

TAREA 12: Tipos de Datos en Bases de Datos

1. Introducción

Los tipos de datos definen la naturaleza de la información que se almacena en una base de datos. Una elección adecuada mejora:

-  **Eficiencia en el almacenamiento**
-  **Rendimiento en consultas**
-  **Validación de la integridad de los datos**

2. Tipos de Datos Fundamentales

2.1. Numéricos

Tipo	Descripción	Ejemplo	Rango Típico
INT	Enteros estándar	edad INT	-2,147,483,648 a 2,147,483,647
SMALLINT	Enteros pequeños	cantidad SMALLINT	-32,768 a 32,767
BIGINT	Enteros grandes	id BIGINT	-9,223,372,036,854,775,808 a 9,223,372,036,854,775,807
DECIMAL(p,s)	Números precisos (p: dígitos, s: decimales)	precio DECIMAL(10,2)	Depende de la precisión definida
FLOAT	Números de punto flotante	temperatura FLOAT	~1.18E-38 a 3.40E+38

2.2. Texto/Cadena

Tipo	Descripción	Ejemplo	Tamaño Máximo
CHAR(n)	Cadena de longitud fija	codigo CHAR(10)	255 caracteres

VARCHAR(n)	Cadena de longitud variable	nombre VARCHAR(100)	65,535 caracteres
TEXT	Texto largo	descripcion TEXT	Hasta 4GB (depende del SGBD)

2.3. Fechas y Horas

Tipo	Descripción	Ejemplo	Formato
DATE	Almacena fecha (año-mes-día)	fecha_nac DATE	YYYY-MM-DD
TIME	Almacena hora (horas:minutos:segundos)	hora TIME	HH:MM:SS
DATETIME	Fecha y hora combinadas	registro DATETIME	YYYY-MM-DD HH:MM:SS
TIMESTAMP	Marca de tiempo (automática en cambios)	ult_actualiz TIMESTAMP	YYYY-MM-DD HH:MM:SS

2.4. Booleanos

Tipo	Descripción	Ejemplo	Valores Válidos
BOOLEAN	Valor lógico (verdadero/falso)	activo BOOLEAN	TRUE/FALSE o 1/0

2.5. Binarios

Tipo	Descripción	Ejemplo	Uso Común
BLOB	Datos binarios grandes (imágenes, PDFs)	archivo BLOB	Almacenamiento de archivos
BINARY(n)	Datos binarios de longitud fija	hash BINARY(32)	Hashes, claves encriptadas

3. Tipos de Datos Avanzados (Dependientes del SGBD)

3.1. PostgreSQL

- **JSON/JSONB:** Almacena datos estructurados en formato JSON (jsonb es más eficiente para consultas).
- **UUID:** Identificadores únicos universales (ej: a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11).

3.2. MySQL/MariaDB

- **ENUM:** Lista de valores predefinidos (ej: estado ENUM('activo', 'inactivo', 'pendiente')).
- **SET:** Conjunto de valores múltiples (ej: permisos SET('lectura', 'escritura', 'ejecucion')).

3.3. SQL Server

- **MONEY:** Para valores monetarios (maneja decimales exactos).
- **XML:** Almacena datos en formato XML.

4. Buenas Prácticas en la Elección de Tipos de Datos

1. **Optimizar el espacio:** Usar SMALLINT en lugar de INT si los valores son pequeños.
2. **Precisión en decimales:** Elegir DECIMAL para cálculos financieros (evitar FLOAT por errores de redondeo).
3. **Textos variables:** Preferir VARCHAR sobre CHAR para ahorrar espacio (a menos que se necesite longitud fija).
4. **Internacionalización:** Usar NVARCHAR (en SQL Server) o VARCHAR con codificación UTF-8 para soportar caracteres multilingües.

5. Ejemplo Práctico: Creación de una Tabla

```
CREATE TABLE empleados (  
    id BIGINT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    email VARCHAR(255) UNIQUE,  
    salario DECIMAL(10, 2),  
    fecha_contratacion DATE,  
    activo BOOLEAN DEFAULT TRUE,  
    foto_perfil BLOB
```

);

6. Referencias

- [MySQL Data Types](#)
- [PostgreSQL Data Types](#)
- [Microsoft SQL Server Data Types](#)